Redflag AI

Implementation Plan — Repo Creation to Deployment & Submission
Native.builder AI Factory Hackathon | Aug 3 – Aug 11, 2026 | 6-Person Team

Two Scope Decisions Locked In For This Plan

1. Bright Data live-web lookup is OUT of scope. It was a nice-to-have in the PRD targeting a partner prize, but it adds an external dependency and integration risk with no MVP payoff. Cutting it protects the 60-second demo budget and the core classify → draft → log pipeline.

2. Drafted-reply tone is FIXED at "neutral-professional" for the MVP — not user-configurable. A firm/friendly/neutral toggle is a UI + prompt-variant surface area the team doesn't need to demo well in 3 minutes. One consistent, professional tone is easier to QA, easier to demo, and still fully satisfies the PRD's core requirement (a usable, editable draft per flagged message). Configurable tone becomes a stretch item only if Day 5-6 buffer allows.

Team Roles Recap

#	Role	Owns
A	Classification Agent Lead	Flag logic: Normal / Scope Creep / Payment Risk + reason
B	Drafting Agent Lead	Suggested reply generation, tone consistency
C	Logging Agent + Data Lead	Per-client log schema, native.builder data table, logging agent
D	Frontend / UI Lead	Paste-in input, flagged list, dashboard, editable draft UI
E	Integration & Deployment Lead	native.builder pipeline wiring, repo, public deployment
F	Product / Demo Lead	Sample conversations, demo script/video, judging-criteria alignment

How We Use native.builder

- **App shell & UI:** native.builder hosts the whole workspace — paste-in box, flagged-message list, editable draft panel, and per-client log view all live as screens/components inside one native.builder app, built by D.
- **Classification + Drafting agents:** built as native.builder's built-in AI agent nodes. Classification agent takes raw pasted text in, returns a structured list of {message, label, reason}. Drafting agent takes a flagged message + label in, returns a draft reply string. These are two separate agent nodes, not one giant prompt — keeps each easy to test and tune independently (A and B each own one).
- **Logging agent:** a third lightweight agent (or native.builder workflow step) that takes a confirmed flag and writes a structured row to native.builder's built-in data/backend layer. C owns the schema and the write logic.
- **Pipeline wiring:** E connects the three agent nodes plus the UI into one linear workflow inside native.builder: paste text → classify → (for each flagged msg) draft → log. This is the "multi-step agent pipeline" the judging criteria explicitly reward over a single prompt-response bot.
- **Data/backend:** per-client log entries are stored in native.builder's native data tables — no external DB needed. Fields: date, client (if multi-client stretch is on), flag_type, summary, reply_sent (bool).
- **Deployment:** native.builder's built-in publish/deploy button produces the public URL required for submission (FR: "Deployed, publicly accessible URL via native.builder"). E triggers this early (Day 2) with a placeholder pipeline so the deploy step is never a Day 10 surprise, then re-deploys as features land.

FR / NFR Reference (used as the checklist basis below)

ID	Requirement	Type
FR1	Paste-in text input for a client conversation	Functional
FR2	Classification agent flags Normal / Scope Creep / Payment Risk + one-line reason	Functional
FR3	Drafting agent produces one editable suggested reply per flagged message	Functional
FR4	Per-client log view: chronological list of flags + outcomes	Functional
FR5	App is deployed to a publicly accessible native.builder URL	Functional
FR6*	Stretch: multi-client profiles, risk score counter (only if Day 5-6 buffer allows)	Functional (stretch)
NFR1	Full workflow (paste → flags → draft → log) completes in under 60 seconds	Performance
NFR2	Correctly classifies at least 3 distinct signal types in one sample conversation	Reliability
NFR3	UI is clean and demo-legible (single before/after view judges can follow)	Usability
NFR4	Agent pipeline is modular (classify / draft / log as separate steps, not one prompt)	Maintainability
NFR5	Deployed URL stays stable/available through judging window	Availability

Day-by-Day Plan (Aug 3 – Aug 11)

Each day lists a goal and a per-member checklist tagged with the FR/NFR it serves. Check items off at end of day — if a box is still open at the next standup, it blocks that member's next day and should be flagged immediately.

Day 1 — Mon, Aug 3 — Repo, native.builder Setup & Foundations

Goal: Get the repo, the native.builder project, and every member's starting artifact in place. No agent-building yet.

E — Integration & Deployment Lead

- ■ Create GitHub repo (README, .gitignore, LICENSE) [supports FR5, NFR4]
- ■ Create the native.builder project/app shell and add all 6 members as collaborators
- ■ Set up shared task board (Trello/Notion/Sheet) mirroring this checklist
- ■ Write down branch/commit convention so no one blocks each other later

A — Classification Agent Lead

- ■ Draft classification taxonomy: exact definitions of Normal / Scope Creep / Payment Risk [FR2]
- ■ Collect 5 real-style flagged example messages (mix of all 3 labels)
- ■ List 3 ambiguous edge cases to test later (e.g. "let's discuss budget later")

B — Drafting Agent Lead

- ■ Write down the tone guideline: neutral-professional, fixed for MVP [locks FR3 scope]
- ■ Draft 2 example replies by hand (1 scope-boundary, 1 payment nudge) as prompt reference examples

C — Logging Agent + Data Lead

- ■ Design log schema: date, client, flag_type, summary, reply_sent [supports FR4]
- ■ Sketch the native.builder data table structure (field names + types)

D — Frontend / UI Lead

- ■ Wireframe the 3-panel dashboard: paste-in box, flagged list, per-client log [NFR3]
- ■ Pick type/color direction (read frontend-design principles before committing)

F — Product / Demo Lead

- ■ Write 3 synthetic client conversation snippets covering all 3 signal types [feeds NFR2]
- ■ Draft demo script v1 using the PRD's 15/30/45/45/30/15-second beats

Day 2 — Tue, Aug 4 — Core Agents: First Working Versions

Goal: Every agent has a first runnable version inside native.builder, even if rough. E confirms deploy works end to end.

A — Classification Agent Lead

- ■ Build classification agent (native.builder AI agent node) [FR2]
- ■ Run it against Day 1's 5 sample messages, record accuracy
- ■ Adjust taxonomy wording based on first misses

B — Drafting Agent Lead

- ■ Build drafting agent node; input = flagged message + reason, output = draft reply [FR3]
- ■ Confirm output matches the neutral-professional tone guideline on 3 test cases

C — Logging Agent + Data Lead

- ■ Create the native.builder data table for logs
- ■ Build logging agent/workflow-step skeleton (no live connection yet)

D — Frontend / UI Lead

- ■ Build the paste-in text input component (static, no agent wired yet)
- ■ Build a static flagged-message list component using dummy data

E — Integration & Deployment Lead

- ■ Wire input → classification agent inside native.builder (first live connection) [NFR4]
- ■ Trigger a placeholder deployment to confirm the public URL pipeline works early [de-risks FR5]

F — Product / Demo Lead

- ■ Expand to 5 total sample conversations
- ■ Get a 5-minute gut-check read from 2 teammates on realism of the samples

Day 3 — Wed, Aug 5 — Pipeline Wiring: Classify → Draft → Log

Goal: Connect all three agents into one linear pipeline. First true end-to-end smoke test happens today.

A — Classification Agent Lead

- ■ Refine prompt against Day 1's 3 ambiguous edge cases
- ■ Standardize the one-line "reason" output format so B/D can parse it reliably

B — Drafting Agent Lead

- ■ Confirm drafting agent correctly consumes classification agent's output format
- ■ Hand off exact output shape (draft text field) to D for the editable UI

C — Logging Agent + Data Lead

- ■ Finish logging agent: auto-writes an entry whenever a flag is produced [FR4]
- ■ Test one full write + read cycle against the data table

D — Frontend / UI Lead

- ■ Build the editable draft-reply UI component (textbox pre-filled, user can edit) [FR3]
- ■ Connect flagged list to real classification agent output (replace dummy data)

E — Integration & Deployment Lead

- ■ Wire classify → draft → log as one native.builder pipeline [NFR4]
- ■ Run the first full end-to-end smoke test with one sample conversation

F — Product / Demo Lead

- ■ Update demo script v2 with judging-criteria call-outs (tech application, business value, originality)
- ■ Time a rough dry run against script v2

Day 4 — Thu, Aug 6 — Per-Client Log View + Full Integration

Goal: The log view goes live and the whole loop runs under real timing pressure for the first time.

A — Classification Agent Lead

- ■ Stress-test classification against all 5 sample conversations end to end
- ■ Tune prompt until at least 3 distinct signal types are correctly caught per NFR2

B — Drafting Agent Lead

- ■ Tone QA pass across scope-creep vs payment-risk replies — check for consistency

C — Logging Agent + Data Lead

- ■ Build the per-client log dashboard view: chronological list of flags + outcomes [FR4]
- ■ Connect the view to the live data table

D — Frontend / UI Lead

- ■ Integrate the log view into the main dashboard alongside input + flagged list
- ■ Polish flagged-list interactions (click to expand reason/draft)

E — Integration & Deployment Lead

- ■ Run full pipeline integration test, paste → flags → draft → log, and time it [NFR1: under 60s]
- ■ Fix any latency bottlenecks found

F — Product / Demo Lead

- ■ Record a rough, non-final internal demo run for team feedback

Day 5 — Fri, Aug 7 — MVP Feature Freeze

Goal: All 5 must-have FRs should work end to end today. No new functional work past this point unless fixing a bug.

A — Classification Agent Lead

- ■ Freeze classification prompt; write down 2-3 known limitations in the README

B — Drafting Agent Lead

- ■ Freeze drafting prompt; document the fixed-tone decision in the README

C — Logging Agent + Data Lead

- ■ Confirm log entries persist correctly across page reloads/new sessions

D — Frontend / UI Lead

- ■ Run a mobile/small-screen check on the dashboard; fix any layout breaks [NFR3]

E — Integration & Deployment Lead

- ■ Call a go/no-go on stretch features (multi-client profiles, risk score) based on remaining time
- ■ Confirm Bright Data stays out of scope (already decided) — no rework here

F — Product / Demo Lead

- ■ Finalize demo script v3 with exact per-beat timings matching the PRD's 3-minute structure

Day 6 — Sat, Aug 8 — Stretch Work (If Capacity) + Hardening

Goal: Only touch FR6 stretch items if the team lead confirmed go on Day 5. Otherwise, this whole day is QA and hardening.

A — Classification Agent Lead

- ■ If no stretch work assigned: pair-test with C/D instead of writing new prompts

B — Drafting Agent Lead

- ■ If time allows: draft a second reply variant per flag type as an optional alternative

C — Logging Agent + Data Lead

- ■ Stretch (if greenlit): implement multi-client profile separation in the log view [FR6*]

D — Frontend / UI Lead

- ■ Stretch (if greenlit): add a simple risk-score counter per client on the dashboard [FR6*]

E — Integration & Deployment Lead

- ■ Monitor pipeline stability under repeated test runs; fix bugs surfaced during pair-testing

F — Product / Demo Lead

- ■ Begin recording final demo video takes

Day 7 — Sun, Aug 9 — Full QA + Deployment Hardening

Goal: Every member tests a flow they did not build. Deployment gets a stability check under repeated use.

A — Classification Agent Lead

- ■ Run 10 new edge-case messages through classification; log any failures

B — Drafting Agent Lead

- ■ Verify every flag type produces a sensible, non-empty draft with no broken formatting

C — Logging Agent + Data Lead

- ■ Verify log accuracy across 3 separate test runs (no duplicate/missing entries)

D — Frontend / UI Lead

- ■ Final UI polish pass; re-check responsive layout on a second device/screen size

E — Integration & Deployment Lead

- ■ Verify the public native.builder URL is stable under back-to-back test submissions [NFR5]

F — Product / Demo Lead

- ■ Edit the demo video to final cut; prepare a backup screen recording in case the live demo hiccups

Day 8 — Mon, Aug 10 — Submission Day

Goal: Final deploy, final sanity pass, and the actual submission goes in. Nothing new gets built today.

E — Integration & Deployment Lead

- ■ Trigger the final native.builder deployment; confirm the public URL is live [FR5]

A — Classification Agent Lead

- ■ Run the exact demo conversation through the live URL once more; confirm flags match expectations

B — Drafting Agent Lead

- ■ Confirm drafts on the live URL still read correctly (no stale cached prompt versions)

C — Logging Agent + Data Lead

- ■ Confirm the log view on the live URL reflects the final test run correctly

D — Frontend / UI Lead

- ■ Final visual sanity check on the live deployed URL, not just local/dev preview

F — Product / Demo Lead

- ■ Finalize submission form text, upload the demo video, double check judging-criteria write-up
- ■ Confirm the submission was received before the deadline

Day 9 — Tue, Aug 11 — Buffer / Contingency Day

Goal: Use only if submission allows late fixes, or otherwise close out the project cleanly.

E — Integration & Deployment Lead

- ■ If deadline was extended: fix any last deployment issue reported by judges/team
- ■ If not: export/back up the repo and native.builder project

F — Product / Demo Lead

- ■ Run a short team retro: what worked, what to reuse next hackathon

All — Whole Team

- ■ Confirm every FR/NFR in the reference table on page 2 is checked off or explicitly logged as a known gap

Final Submission Checklist

- ■ Public native.builder URL is live and loads the full dashboard [FR5, NFR5]
- ■ Paste-in → classification → draft → log completes in under 60 seconds on the live URL [NFR1]
- ■ At least 3 distinct signal types are correctly classified in the demo conversation [NFR2]
- ■ Every flagged message has an editable draft reply, none blank or broken [FR3]
- ■ Per-client log view shows a correct chronological history [FR4]
- ■ 3-minute demo video follows the PRD's beat structure and is uploaded
- ■ README documents: fixed neutral-professional tone decision, Bright Data out-of-scope decision, known limitations
- ■ Submission form text explicitly maps the project to all 4 judging criteria